

AURORA_OS_GUIDE

HelixCode for Aurora OS - Complete User Guide

Table of Contents

1. [Introduction](#)
 2. [System Requirements](#)
 3. [Installation](#)
 4. [Configuration](#)
 5. [Core Features](#)
 6. [Enhanced Security](#)
 7. [System Monitoring](#)
 8. [Native Integration](#)
 9. [User Interface](#)
 10. [Theme Customization](#)
 11. [Troubleshooting](#)
 12. [Best Practices](#)
 13. [FAQ](#)
-

Introduction

HelixCode for Aurora OS is a specialized client optimized for Aurora OS (Russian platform), featuring enhanced security measures, comprehensive system monitoring, and deep native integration with Aurora OS ecosystem features.

Key Features

- **Enhanced Security:** Multi-level security with encryption and access control
 - **System Monitoring:** Deep integration with Aurora OS system metrics
 - **Native Integration:** Optimized for Aurora OS platform features
 - **Security Audit Logging:** Comprehensive audit trails for compliance
 - **Access Control:** Role-based access control with fine-grained permissions
 - **Compliance Framework:** Built-in support for regulatory requirements
-

System Requirements

Minimum Requirements

- **Operating System:** Aurora OS 4.0 or later
- **Architecture:** x86_64 or aarch64
- **RAM:** 2 GB minimum (4 GB recommended)
- **Storage:** 300 MB free space
- **Network:** Active network connection
- **Database:** PostgreSQL 13+

Recommended Requirements

- **Operating System:** Aurora OS 5.0+
- **RAM:** 8 GB or more
- **Storage:** 1 GB free space (for logs and audit trails)
- **Network:** High-speed connection for distributed workers
- **Database:** PostgreSQL 15+ with connection pooling
- **Redis:** Redis 6.0+ for caching (optional but recommended)

Hardware Support

- **CPU:** Multi-core processor (4+ cores recommended)
 - **Storage:** SSD recommended for database performance
 - **Network:** Gigabit Ethernet or better for distributed operations
-

Installation

Quick Installation (Automated)

The easiest way to install HelixCode on Aurora OS is using the automated deployment script:

```
# Build the binary
cd HelixCode
make aurora-os
```

```
# Deploy with automated script (requires root)
sudo ./scripts/deploy-aurora-os.sh
```

The script will: - Install the binary to /opt/helixcode/ - Create default configuration at /etc/helixcode/aurora-config.yaml - Set up systemd

service - Create helixcode user and set appropriate permissions - Configure logging and data directories - Set up security features

Manual Installation

If you prefer manual installation:

Step 1: Build the Binary

```
cd HelixCode
make aurora-os
```

This creates bin/aurora-os (approximately 56 MB).

Step 2: Install Binary

```
sudo mkdir -p /opt/helixcode
sudo cp bin/aurora-os /opt/helixcode/
sudo chmod +x /opt/helixcode/aurora-os
sudo ln -s /opt/helixcode/aurora-os /usr/local/bin/helixcode-aurora
```

Step 3: Create Configuration

```
sudo mkdir -p /etc/helixcode
sudo mkdir -p /var/log/helixcode
sudo mkdir -p /var/lib/helixcode

# Create config file (see Configuration section below)
sudo nano /etc/helixcode/aurora-config.yaml
```

Step 4: Create System User

```
sudo useradd -r -s /bin/false -d /var/lib/helixcode helixcode
sudo chown -R helixcode:helixcode /var/log/helixcode
sudo chown -R helixcode:helixcode /var/lib/helixcode
```

Step 5: Configure Service

For systemd-based systems:

```
sudo cp scripts/helixcode-aurora.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable helixcode-aurora
sudo systemctl start helixcode-aurora
```

Configuration

Basic Configuration

Create `/etc/helixcode/aurora-config.yaml`:

```
# HelixCode Aurora OS Configuration

server:
  address: "0.0.0.0"
  port: 8080

database:
  host: "localhost"
  port: 5432
  user: "helixcode"
  dbname: "helixcode"
  # Set password via HELIX_DATABASE_PASSWORD environment variable

redis:
  enabled: false # Optional, but recommended
  host: "localhost"
  port: 6379

auth:
  token_expiry: 86400 # 24 hours
  session_expiry: 604800 # 7 days

aurora:
  # Security features
  enable_security_features: true
  enable_system_monitoring: true
  enable_native_integration: true
  security_level: "enhanced" # or "standard", "maximum"

  # Audit logging
  audit_logging:
    enabled: true
    log_path: "/var/log/helixcode/audit.log"
    retention_days: 365

  # Access control
  access_control:
```

```
    enabled: true
    enforce_rbac: true

workers:
  health_check_interval: 30
  max_concurrent_tasks: 10

logging:
  level: "info"
  file: "/var/log/helixcode/aurora-os.log"
```

Environment Variables

Create /etc/helixcode/aurora.env:

```
# Database
HELIX_DATABASE_PASSWORD=your_secure_password

# Redis (if enabled)
HELIX_REDIS_PASSWORD=your_redis_password

# JWT Secret
HELIX_AUTH_JWT_SECRET=your_jwt_secret_key

# Aurora OS specific
AURORA_SECURITY_LEVEL=enhanced
AURORA_MONITORING_ENABLED=true
AURORA_NATIVE_INTEGRATION=true
```

Advanced Configuration

Security Settings

```
aurora:
  security:
    # Encryption
    encryption_enabled: true
    encryption_algorithm: "AES-256-GCM"

    # Access control
    access_control:
      enable_ip_whitelist: true
      allowed_ips:
        - "192.168.1.0/24"
        - "10.0.0.0/8"
```

```
enable_rate_limiting: true
rate_limit:
  requests_per_minute: 60
  burst: 10

# Audit logging
audit:
  log_all_requests: true
  log_data_access: true
  log_authentication: true
  log_authorization: true
  sensitive_data_masking: true
```

System Monitoring Settings

```
aurora:
  monitoring:
    enabled: true

# Resource monitoring
resources:
  cpu_threshold: 80
  memory_threshold: 85
  disk_threshold: 90

# Metrics collection
metrics:
  enabled: true
  interval: 30 # seconds
  retention_hours: 168 # 7 days

# Alerting
alerts:
  enabled: true
  notification_channels:
    - email
    - slack
```

Core Features

Starting the Application

```
# Start as system service
sudo systemctl start helixcode-aurora

# Or run directly
helixcode-aurora

# Run with custom config
helixcode-aurora --config /path/to/config.yaml
```

Main Interface

The Aurora OS client provides a native Fyne-based GUI with four main tabs:

1. **Dashboard:** System overview, task status, and resource usage
2. **Tasks:** Task management and execution monitoring
3. **Workers:** Distributed worker pool management
4. **Settings:** Configuration and preferences

Basic Operations

Creating a Task

```
# Via CLI
helixcode-aurora task create \
  --name "Security Analysis" \
  --type "security" \
  --priority "high"

# Via API
curl -X POST http://localhost:8080/api/tasks \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "name": "Security Analysis",
    "type": "security",
    "priority": "high"
  }'
```

Monitoring Task Progress

List all tasks

```
helixcode-aurora task list
```

Get task details

```
helixcode-aurora task get <task-id>
```

Stream task logs

```
helixcode-aurora task logs <task-id> --follow
```

Enhanced Security

Overview

Aurora OS client implements multi-level security measures designed for secure environments and regulatory compliance.

Security Levels

Standard Security

Basic security features:

```
aurora:
  security_level: "standard"
  security:
    encryption_enabled: true
    audit_logging: true
    basic_access_control: true
```

Enhanced Security (Recommended)

Advanced security features:

```
aurora:
  security_level: "enhanced"
  security:
    encryption_enabled: true
    encryption_algorithm: "AES-256-GCM"
    audit_logging: true
    advanced_access_control: true
    rate_limiting: true
```

```
ip_whitelist: true
session_management: "strict"
```

Maximum Security

For high-security environments:

```
aurora:
  security_level: "maximum"
  security:
    encryption_enabled: true
    encryption_algorithm: "AES-256-GCM"
    audit_logging: true
    audit_log_encryption: true
    advanced_access_control: true
    rate_limiting: true
    ip_whitelist: true
    session_management: "strict"
    multi_factor_auth_required: true
    data_loss_prevention: true
    intrusion_detection: true
```

Encryption

Data at Rest

```
aurora:
  security:
    encryption:
      data_at_rest: true
      database_encryption: true
      file_encryption: true
      key_rotation: true
      key_rotation_days: 90
```

Data in Transit

```
server:
  tls_enabled: true
  tls_cert: "/etc/helixcode/cert.pem"
  tls_key: "/etc/helixcode/key.pem"
  tls_min_version: "1.3"
  cipher_suites:
    - TLS_AES_256_GCM_SHA384
    - TLS_CHACHA20_POLY1305_SHA256
```

Access Control

Role-Based Access Control (RBAC)

```
# Create role
helixcode-aurora rbac role create \
  --name admin \
  --permissions read,write,execute,admin

# Assign role to user
helixcode-aurora rbac user assign \
  --user alice \
  --role admin

# List permissions
helixcode-aurora rbac permissions list
```

Fine-Grained Permissions

```
aurora:
  access_control:
    permissions:
      tasks:
        read: ["user", "admin"]
        write: ["admin"]
        delete: ["admin"]
      workers:
        read: ["user", "admin"]
        manage: ["admin"]
      settings:
        read: ["admin"]
        write: ["admin"]
```

Audit Logging

Enable Comprehensive Audit Logging

```
aurora:
  audit:
    enabled: true
    log_path: "/var/log/helixcode/audit.log"

# What to log
log_authentication: true
```

```
log_authorization: true
log_data_access: true
log_configuration_changes: true
log_security_events: true

# Retention and rotation
retention_days: 365
rotation_size: "100MB"
compression: true
```

View Audit Logs

```
# View recent audit events
helixcode-aurora security audit show --recent 100

# Search audit logs
helixcode-aurora security audit search \
  --user alice \
  --action delete \
  --from "2025-01-01"

# Export audit logs
helixcode-aurora security audit export \
  --format json \
  --output /tmp/audit-export.json
```

Security Monitoring

```
# Check security status
helixcode-aurora security status

# Run security scan
helixcode-aurora security scan

# View security alerts
helixcode-aurora security alerts

# Generate security report
helixcode-aurora security report \
  --format pdf \
  --output /tmp/security-report.pdf
```

System Monitoring

Overview

Aurora OS client provides deep integration with system monitoring capabilities, tracking CPU, memory, disk, and network usage.

Real-Time Monitoring

```
# Start real-time monitoring
helixcode-aurora monitor start

# View current metrics
helixcode-aurora monitor status

# View specific resource
helixcode-aurora monitor cpu
helixcode-aurora monitor memory
helixcode-aurora monitor disk
helixcode-aurora monitor network
```

Resource Thresholds

Configure alerts for resource usage:

```
aurora:
  monitoring:
    thresholds:
      cpu:
        warning: 70
        critical: 85
      memory:
        warning: 75
        critical: 90
      disk:
        warning: 80
        critical: 95
```

Metrics Collection

Enable Prometheus Metrics

```
monitoring:
  prometheus:
```

```
enabled: true
endpoint: "/metrics"
port: 9090
```

View Metrics

```
# System metrics
curl http://localhost:9090/metrics
```

```
# CPU usage
helixcode-aurora metrics cpu --duration 1h
```

```
# Memory usage
helixcode-aurora metrics memory --duration 1h
```

```
# Disk I/O
helixcode-aurora metrics disk --duration 1h
```

Historical Data

```
# View historical metrics
helixcode-aurora monitor history \
  --resource cpu \
  --from "2025-01-01" \
  --to "2025-01-07"
```

```
# Export metrics
helixcode-aurora monitor export \
  --format csv \
  --output /tmp/metrics.csv
```

Alerting

Configure alerts for monitoring events:

```
aurora:
  monitoring:
    alerts:
      enabled: true

  rules:
    - name: "High CPU Usage"
      condition: "cpu_usage > 85"
      duration: "5m"
      severity: "critical"
```

```
actions:
  - email
  - slack

- name: "Low Disk Space"
  condition: "disk_usage > 90"
  duration: "1m"
  severity: "critical"
  actions:
    - email
```

Native Integration

Overview

Aurora OS client integrates deeply with Aurora OS platform features for optimal performance and user experience.

Platform Features

Aurora OS System API

```
# Check Aurora OS version
helixcode-aurora platform version

# Get system information
helixcode-aurora platform info

# Check platform features
helixcode-aurora platform features
```

Native Services Integration

```
aurora:
  native_integration:
    enabled: true

  # Aurora OS specific services
  services:
    - name: "aurora_security"
      enabled: true
    - name: "aurora_monitoring"
      enabled: true
```

```
- name: "aurora_filesystem"  
  enabled: true
```

Performance Optimization

Aurora OS-specific optimizations:

```
aurora:  
  optimization:  
    # Platform-specific optimizations  
    use_native_threading: true  
    use_native_networking: true  
    use_native_file_system: true  
  
    # Performance tuning  
    thread_pool_size: 16  
    network_buffer_size: 65536  
    io_buffer_size: 131072
```

User Interface

Dashboard Tab

The dashboard provides an overview of:

- **System Status:** CPU, memory, disk usage
- **Security Status:** Current security level, recent security events
- **Active Tasks:** Currently executing tasks with progress
- **Worker Pool:** Connected distributed workers
- **Recent Activity:** Last 10 operations
- **Alerts:** System warnings, security alerts, and errors

Tasks Tab

Task management interface with:

- **Task List:** All tasks with status, priority, and progress
- **Filter/Search:** Filter by status, type, priority
- **Task Details:** Detailed view of selected task
- **Security Context:** Security level and audit trail for each task
- **Actions:** Create, cancel, retry, delete tasks

Workers Tab

Worker pool management:

- **Worker List:** All connected workers with status
- **Worker Details:** Capabilities, resource usage
- **Connection Status:** Network health
- **Security Status:** Worker authentication status
- **Actions:** Add, remove, configure workers

Settings Tab

Configuration interface:

- **General:** Basic application settings
 - **Security:** Security level and policies
 - **Monitoring:** System monitoring configuration
 - **Theme:** Visual appearance customization
 - **Advanced:** Expert settings
-

Theme Customization

Aurora Theme

The default Aurora theme uses cool colors optimized for Aurora OS:

- **Primary:** Cool Cyan (#00D4FF)
- **Secondary:** Blue (#0099CC)
- **Accent:** Red/Pink (#FF6B6B)
- **Background:** Very Dark Blue-Black (#0F1419)
- **Border:** Dark Blue (#1E3A5F)

Switching Themes

Via CLI

```
helixcode-aurora theme set aurora
```

Available themes

```
helixcode-aurora theme list
```

Output: Dark, Light, Helix, Aurora

Custom Theme

Create a custom theme:

```
theme:
  name: "MyTheme"
  is_dark: true
  colors:
    primary: "#YOUR_COLOR"
    secondary: "#YOUR_COLOR"
    accent: "#YOUR_COLOR"
    text: "#FFFFFF"
    background: "#000000"
    border: "#333333"
    success: "#00FF88"
    warning: "#FFB347"
    error: "#FF4757"
    info: "#3742FA"
```

Load custom theme:

```
helixcode-aurora theme load /path/to/theme.yaml
```

Troubleshooting

Common Issues

Issue: Application Won't Start

Symptoms: Service fails to start, immediate exit

Solutions:

1. Check configuration file syntax:

```
helixcode-aurora config validate
```

2. Verify database connection:

```
psql -h localhost -U helixcode -d helixcode
```

3. Check logs:

```
sudo journalctl -u helixcode-aurora -n 50
```

4. Verify Aurora OS compatibility:

```
cat /etc/aurora-release
```

Issue: Security Features Not Working

Symptoms: Encryption disabled, audit logs not generated

Solutions:

1. Verify security level:

```
helixcode-aurora security status
```

2. Check security configuration:

```
helixcode-aurora config show | grep security
```

3. Review security logs:

```
tail -f /var/log/helixcode/security.log
```

4. Run security diagnostics:

```
helixcode-aurora security diagnose
```

Issue: High Memory Usage

Symptoms: Application consuming excessive memory

Solutions:

1. Enable memory optimization:

```
aurora:  
  optimization:  
    memory_optimization: true  
    max_memory: "2GB"
```

2. Clear caches:

```
helixcode-aurora cache clear
```

3. Reduce concurrent tasks:

```
workers:  
  max_concurrent_tasks: 5
```

Diagnostic Commands

```
# System diagnostics  
helixcode-aurora diagnostics run
```

```
# Health check
helixcode-aurora health

# Configuration test
helixcode-aurora config test

# Network test
helixcode-aurora network test

# Database connection test
helixcode-aurora db test

# Security test
helixcode-aurora security test
```

Debug Mode

Enable debug logging:

```
logging:
  level: "debug"
  file: "/var/log/helixcode/aurora-debug.log"
```

Or via environment variable:

```
HELIX_LOG_LEVEL=debug helixcode-aurora
```

Best Practices

Security Best Practices

1. Use Maximum Security Level for Sensitive Data

```
aurora:
  security_level: "maximum"
```

2. Enable All Audit Logging

```
aurora:
  audit:
    log_authentication: true
    log_authorization: true
    log_data_access: true
    log_security_events: true
```

3. Regular Security Audits

```
# Weekly security scan
helixcode-aurora security scan

# Monthly security report
helixcode-aurora security report --month $(date +%Y-%m)
```

4. Keep Audit Logs for Compliance

```
aurora:
  audit:
    retention_days: 365 # 1 year minimum
    compression: true
    backup_enabled: true
```

Performance Optimization

1. Enable Redis for Caching

```
redis:
  enabled: true
```

2. Optimize Database

```
-- Regular maintenance
VACUUM ANALYZE;
REINDEX DATABASE helixcode;
```

3. Use Native Optimizations

```
aurora:
  optimization:
    use_native_threading: true
    use_native_networking: true
```

Monitoring Best Practices

1. Set Appropriate Thresholds

```
aurora:
  monitoring:
    thresholds:
      cpu: { warning: 70, critical: 85 }
      memory: { warning: 75, critical: 90 }
```

2. Regular Metrics Review

```
# Daily metrics check
helixcode-aurora monitor status

# Weekly trend analysis
helixcode-aurora metrics analyze --duration 7d
```

3. Enable Alerting

```
aurora:
  monitoring:
    alerts:
      enabled: true
```

Maintenance

1. Regular Log Rotation

```
# Configure logrotate
sudo nano /etc/logrotate.d/helixcode
```

2. Monitor Disk Usage

```
du -sh /var/lib/helixcode/*
du -sh /var/log/helixcode/*
```

3. Database Backups

```
# Daily backup
pg_dump helixcode > backup-$(date +%Y%m%d).sql
```

FAQ

General Questions

Q: What is the difference between Aurora OS client and standard desktop client?

A: The Aurora OS client includes specialized features: - Enhanced multi-level security - Deep system monitoring integration - Aurora OS-specific optimizations - Compliance-ready audit logging - Native platform integration

Q: Can I run the Aurora OS client on non-Aurora OS systems?

A: Yes, but some features will be limited: - Native Aurora OS API integration unavailable - Some optimizations may not apply - Platform-specific features disabled

Q: How much resources does the client consume?

A: Typical resource usage: - RAM: 45-60 MB idle, up to 150 MB under load - CPU: 5-15% average, 30-40% during intensive tasks - Storage: ~56 MB binary + logs and data

Security Questions

Q: What security certifications does Aurora OS client support?

A: The client provides features for: - GOST R standards compliance - Federal security requirements - Industry-specific regulations - Audit trail requirements

Q: How is sensitive data encrypted?

A: Using AES-256-GCM encryption for data at rest and TLS 1.3 for data in transit.

Q: Can audit logs be tampered with?

A: No, audit logs are: - Append-only - Cryptographically signed - Stored with integrity checks - Regularly backed up

Monitoring Questions

Q: What metrics are collected?

A: System metrics include: - CPU usage (per core and average) - Memory usage (RAM and swap) - Disk usage and I/O - Network traffic - Application-specific metrics

Q: How long are metrics retained?

A: Default retention is 7 days, configurable up to 1 year.

Q: Can I export metrics to external monitoring systems?

A: Yes, supports: - Prometheus metrics endpoint - JSON export - CSV export - Integration with Grafana

Troubleshooting

Q: Why is security level not applying?

A: Common causes: - Configuration not reloaded after changes - Insufficient permissions - Missing dependencies - Check: `helixcode-aurora security diagnose`

Q: Tasks are stuck in “pending” status

A: Check: - Worker availability: `helixcode-aurora workers list` - Task queue: `helixcode-aurora task queue` - Database connectivity - System resources

Q: How do I reset to factory defaults?

A:

```
sudo systemctl stop helixcode-aurora
sudo rm -rf /var/lib/helixcode/*
sudo cp /etc/helixcode/aurora-config.yaml.default /etc/helixcode/aurora-config.yaml
sudo systemctl start helixcode-aurora
```

Additional Resources

Documentation

- [API Reference](#)
- [Architecture Guide](#)
- [Configuration Reference](#)
- [Development Guide](#)
- [Quick Start Guide](#)
- [Deployment Guide](#)

Support

- **GitHub Issues:** <https://github.com/helixcode/helixcode/issues>
- **Community Forum:** <https://forum.helixcode.dev>
- **Documentation:** <https://docs.helixcode.dev>

Related Guides

- [Harmony OS Guide](#) - For Harmony OS platform
 - [Deployment Guide](#) - Production deployment
-

Document Version: 1.0.0 **Last Updated:** 2025-11-07 **HelixCode Version:** 1.0.0+
Platform: Aurora OS 4.0+